

Kanban API

Guia completo de estudo, demonstração e apresentação técnica

Java 21 · Spring Boot · PostgreSQL · Flyway · Docker · Testcontainers · OpenAPI

Objetivo: explicar o sistema com segurança em uma entrevista técnica, incluindo decisões, regras, testes e trade-offs.

Repositório: github.com/AlvaroMGueiros/kanban-api

Validação: 71 testes unitários + 44 testes de integração/API.

1. A apresentação em 60 segundos

“Eu desenvolvi uma API REST para gerenciar projetos em um quadro Kanban. O ponto central é que o status não é salvo diretamente: ele é derivado das datas previstas e realizadas, evitando inconsistências com a passagem do tempo. Separei regras de status, métricas e transições em serviços de domínio testáveis, usei PostgreSQL com Flyway, filtros paginados no banco, tratamento padronizado de erros, Swagger, Docker e CI. A solução tem testes unitários fortes nas regras e testes de integração com PostgreSQL real via Testcontainers.”

O que enfatizar

- O desafio é mais sobre regras temporais e consistência do que sobre CRUD.
- Status é consequência das datas; não existe `setStatus`.
- Transições são transacionais e verificam o estado final.
- Filtros acontecem antes da paginação no PostgreSQL.
- As ambiguidades foram documentadas e cobertas por testes.

2. Visão geral funcional

O sistema gerencia duas entidades principais:

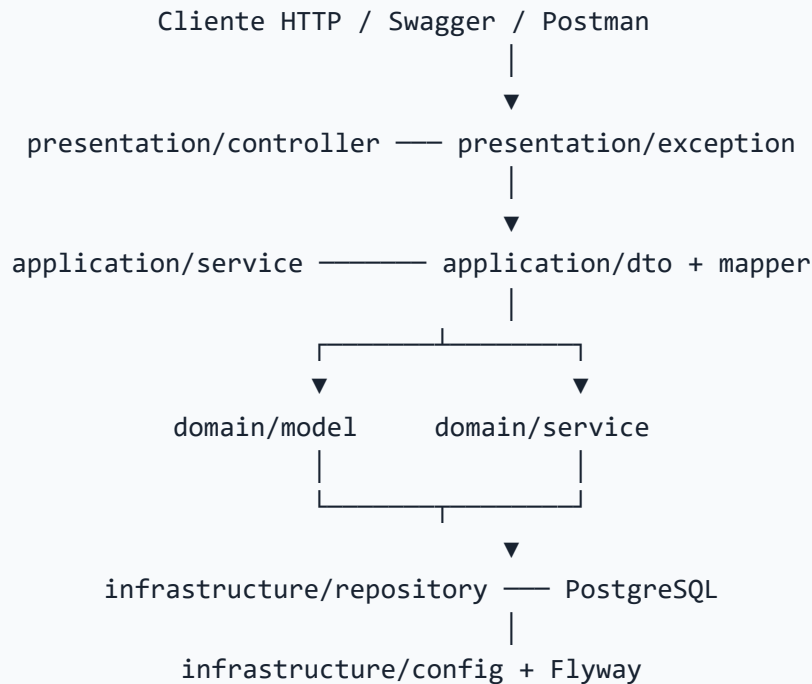
- **Responsável:** nome, e-mail único, cargo, secretaria e auditoria.
- **Projeto:** nome, responsáveis, cronograma, métricas e status calculado.

Um projeto pode ter vários responsáveis e um responsável pode participar de vários projetos. A API oferece CRUDs, listagens paginadas, filtros, quadro Kanban, transições e indicador de quantidade por status.

Funcionalidades entregues

Área	Entrega
Responsáveis	CRUD, validação, e-mail normalizado e único, bloqueio de exclusão quando vinculado.
Projetos	CRUD, múltiplos responsáveis, datas, auditoria, versionamento otimista.
Kanban	Filtro por coluna e transição validada entre estados.
Métricas	Dias de atraso e percentual de tempo restante.
Consultas	Paginação, ordenação e filtros combináveis.
Indicadores	Quantidade de projetos por cada status.
Entrega	Swagger, Postman, Docker, CI, ADRs, diagramas e AI_USAGE.

3. Arquitetura



Responsabilidade de cada camada

Camada	Responsabilidade	Exemplos
Presentation	Contrato HTTP, parâmetros, validação de entrada e erros.	ProjectController , ApiExceptionHandler
Application	Orquestra caso de uso, transação, banco e mapeamento.	ProjectService , KanbanService
Domain	Invariantes e regras que podem ser testadas sem Spring.	ProjectStatusPolicy , KanbanTransitionService
Infrastructure	Persistência, consultas e configuração técnica.	ProjectRepository , ProjectSpecifications

Por que uma arquitetura pragmática?

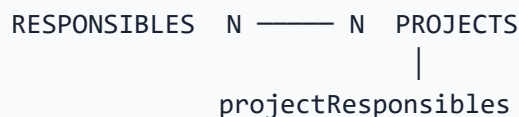
O projeto não cria interfaces ou modelos duplicados sem necessidade. As entidades têm anotações JPA, mas as regras centrais continuam isoladas. Isso reduz cerimônia e mantém testabilidade. Para o tamanho do desafio, separar um domínio totalmente independente da persistência duplicaria modelos e mapeamentos sem benefício proporcional.

Fluxo de uma requisição

1. O controller recebe um DTO e o Bean Validation verifica formato e obrigatoriedade.
2. O serviço captura `LocalDate.now(clock)` uma vez.
3. O serviço carrega entidades e chama as regras de domínio.
4. O domínio valida datas, calcula estado ou executa uma transição.

5. O repositório persiste dentro da transação.
6. O mapper produz o DTO de resposta com métricas calculadas.
7. Exceções são convertidas para o contrato HTTP padronizado.

4. Modelo e banco de dados



A relação N:N é suficiente porque o vínculo não possui atributos próprios. Caso fosse necessário registrar função no projeto, percentual de alocação ou período, a tabela de vínculo deveria virar uma entidade.

Decisões de persistência

- Long com identity para IDs.
- Flyway é o único responsável por alterar o schema.
- Hibernate usa `ddl-auto=validate`.
- E-mail normalizado para minúsculas e protegido por constraint única.
- Constraints garantem ordem das datas também no banco.
- `@Version` protege o projeto contra escritas concorrentes silenciosas.
- Exclusão do projeto remove vínculos; responsável vinculado usa `ON DELETE RESTRICT`.

Migrations

Versão	Finalidade
V1	Cria o schema <code>kanban</code> .
V2	Cria responsáveis e unicidade de e-mail.
V3	Cria projetos, vínculo N:N, constraints e índice do responsável.
V4	Adiciona índices para secretaria e datas usadas pelos filtros de status.

5. Regra de status

A precedência é necessária porque mais de uma condição pode ser verdadeira ao mesmo tempo.

Prioridade	Status	Condição
1	CONCLUIDO	<code>actualEndDate != null</code>
2	ATRASADO	Início previsto passou sem início realizado, ou término previsto passou sem conclusão.
3	EM_ANDAMENTO	Início realizado preenchido, sem conclusão e sem atraso.
4	A_INICIAR	Demais casos válidos.

Por que não persistir o status?

Um projeto pode ficar atrasado à meia-noite sem nenhuma alteração no banco. Se o status fosse uma coluna, seria necessário um job de sincronização e ainda existiria uma janela de inconsistência. Derivar o estado garante que a leitura reflita a data atual.

Uso de Clock

Não se espalha `LocalDate.now()` pelo código. Um `Clock` configurável define a referência temporal. Nos testes ele é fixo, o que torna cenários de ontem, hoje, amanhã e virada de fuso determinísticos.

Ambiguidade do vencimento: o texto diz que EM_ANDAMENTO exige término previsto maior que hoje, mas não define nenhum status para um projeto iniciado no próprio dia do prazo. A decisão foi mantê-lo em andamento até o fim desse dia e classificá-lo como atrasado no dia seguinte. Isso evita um registro válido passar a provocar erro apenas pela mudança da data.

Projeto iniciado sem término previsto é rejeitado, pois não há como satisfazer a definição oficial de EM_ANDAMENTO e ele também não está atrasado.

6. Métricas

Dias de atraso

```
se concluído: 0
se terminoPrevisto existe e terminoPrevisto < hoje:
    dias = diasEntre(terminoPrevisto, hoje)
caso contrário: 0
```

Um projeto pode estar atrasado pelo início e ainda ter zero dias de atraso, porque a fórmula oficial mede somente o atraso do término.

Percentual restante

```
total = diasEntre(inicioPrevisto, terminoPrevisto)
restante = diasEntre(hoje, terminoPrevisto)
percentual = restante / total * 100
```

- Sem as duas datas: zero.
- Duração menor ou igual a zero: zero.
- Concluído: zero.
- Resultado limitado entre 0 e 100.
- Duas casas decimais com `HALF_UP`.

7. Transições do Kanban

O endpoint não atribui diretamente um status. Ele recebe o destino, aplica apenas a ação prevista, recalcula e confirma o resultado. A operação inteira está em uma transação.

Origem → destino	Ação e resultado
A_INICIAR → EM_ANDAMENTO	Define início realizado como hoje; exige cronograma compatível.
A_INICIAR → ATRASADO	Não fabrica atraso; bloqueia e orienta ajuste quando ainda não está atrasado.
A_INICIAR → CONCLUIDO	Define término realizado como hoje.
EM_ANDAMENTO → A_INICIAR	Remove início realizado; rejeita se o resultado for atrasado.
EM_ANDAMENTO → ATRASADO	Sem ação automática; exige datas que realmente produzam atraso.
EM_ANDAMENTO → CONCLUIDO	Define término realizado como hoje.
ATRASADO → A_INICIAR	Sem ação automática; orienta correção das datas.
ATRASADO → EM_ANDAMENTO	Sem ação automática; orienta prazo e início realizado.
ATRASADO → CONCLUIDO	Define término realizado como hoje.
CONCLUIDO → A_INICIAR	Bloqueia sem mutação; exige edição explícita das datas.
CONCLUIDO → EM_ANDAMENTO	Remove término realizado; bloqueia se resultar em atraso.
CONCLUIDO → ATRASADO	Remove término realizado somente se o cálculo resultar em atraso.

Atomicidade

Uma exceção causa rollback. Há teste de API que tenta mover um concluído para A_INICIAR, recebe 422 e depois confirma que o término realizado continua salvo. Isso demonstra que não existe efeito parcial.

8. Contrato REST

Método	Rota	Uso
POST / GET	/api/responsibles	Criar e listar responsáveis.
GET / PUT / DELETE	/api/responsibles/{id}	Consultar, editar e excluir.
POST / GET	/api/projects	Criar e listar projetos.
GET / PUT / DELETE	/api/projects/{id}	Consultar, editar e excluir.
GET	/api/kanban/projects?status=...	Consultar coluna do Kanban.
PATCH	/api/kanban/projects/{id}/status	Solicitar uma transição.
GET	/api/indicators/projects-by-status	Quantidade por status.

Filtros

GET /api/projects aceita status, responsibleId, department e text, além de page, size e sort. Os filtros são combináveis.

A regra de status foi traduzida para uma JPA Specification. Assim o PostgreSQL filtra antes de aplicar offset e limite, preservando totalElements e totalPages.

Erros

HTTP	Quando
400	JSON, parâmetro, paginação ou Bean Validation inválidos.
404	Projeto ou responsável não encontrado.
409	E-mail duplicado ou responsável ainda vinculado.
422	Regra de domínio, datas ou transição incompatível.

```
{
  "timestamp": "2026-09-22T15:00:00Z",
  "status": 422,
  "error": "TRANSITION_NOT_ALLOWED",
  "message": "Mensagem específica com o ajuste necessário",
  "path": "/api/kanban/projects/1/status"
}
```

9. Testes

A estratégia prioriza regras de negócio em vez de cobertura artificial.

Tipo	O que prova
Unidade	Precedência, limites temporais, métricas, nulos, doze transições e invariantes.
Serviço	Normalização, duplicidade, recálculo pela passagem do tempo e fuso.
Integração	PostgreSQL real, migrations, constraints, transações, rollback e índices.
API	Contratos, códigos HTTP, validação, filtros, paginação e OpenAPI.

Resultado atual: 71 testes unitários e 44 testes de integração/API, todos aprovados por `mvnw verify`.

Por que Testcontainers?

H2 poderia esconder diferenças de SQL, constraints, schema, precisão temporal e comportamento do PostgreSQL. Testcontainers executa a integração no mesmo banco usado pela aplicação.

10. Docker, execução e CI

```
docker compose up --build --wait
./mvnw verify
```

- Dockerfile multi-stage: JDK no build e JRE no runtime.
- Aplicação executada por usuário sem privilégios.
- PostgreSQL e API possuem healthchecks.
- Compose aguarda o banco saudável antes de iniciar a aplicação.
- GitHub Actions usa Java 21, cache Maven e executa `mvn verify`.

11. Roteiro de demonstração

1. Abra `http://localhost:8080/swagger-ui`.
2. Mostre `GET /actuator/health` e confirme `UP`.
3. Crie um responsável e mostre o e-mail normalizado.
4. Crie um projeto futuro e mostre `A_INICIAR` e percentual restante.
5. Use `PATCH` para mover a `EM_ANDAMENTO` e mostre o início preenchido.
6. Conclua o projeto e mostre término realizado e métricas zeradas.
7. Tente uma transição inválida e destaque a mensagem e o rollback.
8. Mostre filtro por status e paginação.
9. Abra o indicador e mostre as chaves do enum.
10. Finalize mostrando testes, migrations, Docker e workflow do GitHub.

Comandos úteis

```
# Subir
docker compose up --build --wait

# Testar
.\mvnw.cmd verify

# Ver containers
docker compose ps

# Parar preservando dados
docker compose down
```

12. Perguntas prováveis da entrevista

Por que não salvar o status?

Porque ele depende de “hoje”. Um projeto pode mudar para atrasado sem escrita. Derivar evita sincronização por job e elimina a janela de status desatualizado.

Por que status aparece também em uma Specification?

A política de domínio decide o estado de um projeto. Para filtrar antes da paginação, a mesma lógica precisa ser representada na consulta. Testes comparam os comportamentos e impedem divergência silenciosa.

Por que ManyToMany?

O vínculo não possui atributos. Criar uma entidade intermediária seria cerimônia. Se participação ganhar função, período ou alocação, a modelagem deve evoluir.

Por que 422?

A requisição é sintaticamente válida, mas não pode ser processada por causa das regras atuais do projeto. 400 fica para formato e parâmetros; 409 para conflitos de recursos.

Como evita alteração parcial em transição?

O caso de uso usa uma transação. A regra produz um novo cronograma e valida o destino antes do flush. Exceção provoca rollback, comprovado por integração.

Por que Clock?

Torna tempo uma dependência explícita, permite testes determinísticos e resolve o fuso de negócio sem chamadas espalhadas a `now()`.

Por que não usar Lombok ou MapStruct?

Records e mapeadores pequenos já resolvem o problema. As dependências adicionariam geração de código sem ganho proporcional.

Como tratou concorrência?

Projeto possui versão otimista. Isso impede que duas atualizações concorrentes sobrescrevam silenciosamente a mesma versão.

Qual foi o maior trade-off?

Status derivado melhora consistência temporal, mas torna o filtro SQL mais complexo. Preferi esse custo à manutenção de um status persistido que poderia ficar desatualizado.

O que faria com mais tempo?

Autenticação e autorização, secretaria como entidade, histórico de transições, métricas históricas, observabilidade, testes de carga e consulta agregada do indicador.

13. Decisões revisadas durante o trabalho

- **Filtro após paginação:** rejeitado porque quebrava totais e poderia produzir páginas vazias.
- **CONCLUIDO** → **A_INICIAR**: ajustado após releitura da tabela oficial; agora não altera datas automaticamente.
- **Precisão de Instant:** normalizada para microssegundos por compatibilidade com PostgreSQL.
- **Prazo no dia atual:** documentado como em andamento por falta de um estado definido no enunciado.

14. Limitações que você deve admitir

- Não há autenticação ou autorização.
- Secretaria ainda é texto, não entidade.
- Não há histórico de mudanças de status.
- O indicador realiza quatro contagens; grande escala justificaria agregação específica.
- Busca por substring não tem índice trigram.
- Actuator expõe somente saúde.
- Não foram implementados GraphQL, UI ou camada de IA, pois eram diferenciais.

Apresentar limitações demonstra maturidade. Explique por que elas foram deixadas fora: preservar foco no núcleo obrigatório, evitar overengineering e manter a solução explicável.

15. Checklist antes da entrevista

- Executar `docker compose up --build --wait`.
- Confirmar Swagger, health e indicadores.
- Executar `.\mvnw.cmd verify`.
- Conhecer a precedência dos quatro status sem consultar código.
- Explicar pelo menos três transições válidas e três bloqueadas.
- Explicar por que filtro de status precisa ocorrer antes da paginação.
- Explicar ManyToMany e quando ele deixaria de ser adequado.
- Explicar Flyway versus `ddl-auto`.
- Conhecer as limitações e próximos passos.
- Não decorar frases: demonstrar o raciocínio por trás das decisões.

16. Encerramento sugerido

“Eu priorizei a parte mais arriscada do desafio: consistência entre datas, status e transições. Mantive as regras isoladas e determinísticas, validei o comportamento tanto em unidade quanto com PostgreSQL real e documentei as ambiguidades. A solução é simples para o escopo atual, mas possui caminhos claros de evolução.”

Material de estudo gerado a partir do estado validado do repositório Kanban API. Consulte README, ADRs, Swagger e testes como fontes técnicas definitivas.